The Enumeration Bottleneck and Self-Generated Descriptions

Executive Summary

This document addresses two fundamental questions about language-guided program synthesis:

1. **Why can't perfect language guidance compensate for missing higher-level primitives?**
   - Answer: Language affects *primitive probabilities*, but search is over *program compositions*. Even with optimal primitive weights, the model must enumerate through an exponentially larger space of compositions.
2. **Can we train description generation independently of program synthesis?**
   - Answer: Yes, in principle. This connects to the **self-explanation** literature in cognitive science and offers a more cognitively realistic model than LAPS/LILO. However, careful architectural choices are needed to prevent training collapse.

Part I: The Enumeration Bottleneck

1.1 The Core Problem

Your intuition is: *"If the translation model tells us 'left half' → `take(div(length(h), 2), h)` with high probability, shouldn't that be enough?"*

The answer is no, and here's why:

1.2 What Language Guidance Actually Does

The recognition network outputs a **vector of primitive log-probabilities**:

```
R_ (task) → [log P(take), log P(div), log P(length), log P(h), log P(2), ...]
```

If language says "left half is important," we can boost:

```
log P(take) = -0.25   (boosted from -1.0)
log P(div)  = -0.25   (boosted from -1.0)
log P(length) = -0.25 (boosted from -1.0)
log P(2) = -0.25      (boosted from -1.0)
```

1.3 What Enumeration Actually Does

DreamCoder's best-first enumeration scores programs by **summing** log-probabilities:

```
Program score = Σ log P(primitive_i)
```

Programs are explored in order of decreasing score (best first).

1.4 The Critical Insight: Composition ≠ Selection

Knowing which primitives to use is NOT the same as knowing how to compose them.

Consider all type-valid programs using `take`, `div`, `length`, `h`, `2`:

Program	Score	Correct?
`take(length(h), h)`	-0.75	(takes full length)
`take(div(h, 2), h)`	-0.75	(type error)
`div(length(h), take(2, h))`	-1.00	(wrong composition)
`take(div(2, length(h)), h)`	-1.00	(wrong order)
`take(div(length(h), 2), h)`	-1.00	(correct!)

All programs using the "right" primitives have similar scores, but only **one specific composition** is correct.

1.5 Formal Analysis

Theorem (Composition Enumeration Cost):

Let `P_high` be a program using a single primitive of complexity `n`. Let `P_low` be a semantically equivalent program using `k` primitives.

Even with optimal primitive probability assignment, the expected enumeration cost for `P_low` is:

```
E[cost(P_low)] / E[cost(P_high)] = O(b^(k-1))
```

where `b` is the branching factor (number of type-compatible primitive choices per hole).

Concrete numbers: - With `first_half` primitive: ~10 programs enumerated before solution - Without (4 primitives): ~10,000 programs enumerated before solution - **Ratio: ~1,000×**

1.6 Visual Representation

```
                    SEARCH TREE (best-first order)


Score -0.25    [h] [2] [3] [4] ...

Score -0.50    [length(h)] [take(?,h)] [div(?,?)] ...

Score -0.75    [take(length(h),h)] [take(div(?,?),h)] [div(length(h),?)] ...

Score -1.00    [take(div(length(h),?),h)] [take(div(?,length(h)),h)] ...

Score -1.25    [take(div(length(h),2),h)] ← SOLUTION (finally!)



        ALL higher-scored programs must be enumerated FIRST
```

1.7 The Architectural Separation

The bottleneck is fundamental to the architecture:

Component	What It Controls	What It Cannot Control
Language Model	Which primitives to boost	How to compose them
Recognition Network	Primitive log-probabilities	Program tree structure
Enumeration	Search order	(Given by scores)

There is no path from "boost these primitives" to "compose them this way."

1.8 What Would Actually Help?

To truly compensate for missing `first_half`, we would need:

1. **Template guidance**: "The structure is `take(div(length(?), ?), ?)`"
   - But this requires knowing the composition—which is what we're searching for!
2. **Non-additive scoring**: Bonus for "coherent" primitive combinations
   - Breaks PCFG assumptions, makes enumeration intractable

3. **Learned abstractions**: Create `first_half` through library learning
   - This IS the solution—but requires having seen the pattern before

Bottom line: Language guidance is valuable but cannot replace library learning for complex compositions.

Part II: Self-Generated Descriptions

2.1 Your Proposal

You propose learning a description generator that converts observations to language:

```
Hands (X , X ) → Description Generator G_  → Description d
                                                    ↓
                                  Recognition Network R_  ← Examples
                                                    ↓
                                  Primitive Weights → Enumeration
```

This differs from: - **LAPS**: Descriptions provided by humans at test time - **LILO**: LLM generates descriptions for learned abstractions

Your proposal: **Learn to generate descriptions from observations**

2.2 The Cognitive Parallel

This maps directly onto **self-explanation** in cognitive science:

Cognitive Process	Computational Analog
Observing examples	Processing X , X
Spontaneous verbalization	Description generator G_
Primed hypotheses	Weighted primitives from R_
Guided reasoning	Biased enumeration
Solution discovery	Finding program *

Key findings from Chi et al. (1994) and subsequent work: - Self-explanation converts implicit patterns to explicit hypotheses - Generating explanations is more effective than receiving them - Verbalization reveals gaps and triggers corrective learning

2.3 Formal Architecture

Definition (Self-Description System):

$P(\mid X , X) = \Sigma_d P(d \mid X , X) \cdot P(\mid d, X , X)$

Where: - $P(d \mid X , X)$ = Description generator G_ - $P(\mid d, X , X)$ = Recognition-guided enumeration

In practice, use point estimates:

$\hat{d} = G_(X , X)$
$\hat{} = \text{argmax}_ P(\mid \hat{d}, X , X ; R_)$

2.4 Training the Description Generator

Three options for training G_ :

Option A: Supervised from Human Descriptions

```
L_A( ) = -Σ_i log P_{G_ }(d_i* | X_i , X_i )
```

- Requires human annotations
- Descriptions grounded in human intuitions
- Limited scalability (you have 45 rules)

Option B: Reinforcement Learning from Synthesis Success

```
L_B( ) = -E_{d~G_ } [R( *,  )]
```

- No human labels needed
- Descriptions optimized for synthesis utility
- High variance, credit assignment difficult

Option C: Contrastive/Reconstruction Learning

```
L_C( ) = L_contrast +   · L_language
```

- Self-supervised
- Descriptions capture discriminative information
- May not align with synthesis primitives

Recommended: Hybrid curriculum 1. Pre-train G_ with Option C (self-supervised) 2. Fine-tune with Option A (limited human descriptions) 3. Refine with Option B (RL from synthesis)

2.5 Training Independence

Can we train description generation SEPARATELY from program synthesis?

Yes, but with important caveats:

Approach	Pros	Cons
Separate training	Modular, interpretable	Descriptions may not align with synthesis
Joint training	Optimal alignment	Harder optimization (discrete bottleneck)
Pre-train then fine-tune	Best of both	Requires curriculum design

Key requirement for separate training: The description vocabulary must be grounded in primitives. If descriptions use words like "sorted", "pair", "flush", these must map to primitives `is_sorted`, `has_pair`, `uniform_suit`.

2.6 Comparison to LAPS and LILO

Aspect	LAPS	LILO	Self-Description (Your Proposal)
Description source	Human at test time	LLM for abstractions	Learned generator
What's described	The solution program	Learned library entries	The observed examples
Training data	Program-description pairs	None (zero-shot LLM)	Example-description pairs

Aspect	LAPS	LILO	Self-Description (Your Proposal)
Inference	Requires human	Uses LLM	Fully automatic
Cognitive model	Receiving instruction	External knowledge	Self-explanation

Unique contribution of your proposal: 1. **Learns** the description generator (vs. external source) 2. **Describes observations** (vs. programs/abstractions) 3. **Automatic inference** (no human or LLM at test time) 4. **Models self-explanation** (cognitively realistic)

2.7 Critical Challenges

Challenge 1: Training Collapse Without constraints, G_ could learn to produce arbitrary codes:

```
G_ (X) = hash(X)  # Not language, just encoding
```

Mitigations: - Pre-train G_ on language (frozen BERT + fine-tuning) - Force discrete token sampling - Add language model loss for fluency

Challenge 2: Information Bypass If R_ receives hand embeddings directly, it might ignore descriptions.

Mitigations: - Don't feed raw hands to R_ —only feed (description, labels) - Use auxiliary loss: predict description from primitives

Challenge 3: Sample Complexity 45 rules may be insufficient for learning compositional description generation.

Mitigations: - Pre-train G_ on general language corpus - Use standardized vocabulary (fewer patterns to learn) - Data augmentation (paraphrase descriptions)

Part III: Implementation Recommendations

3.1 Recommended Architecture

```
                    TRAINING PIPELINE


   Phase 1: Pre-train G_

   X , X  → ContrastiveEncoder → z → LanguageDecoder → d
   Loss: Contrastive + Language Model

   Phase 2: Pre-train R_  with human/LLM descriptions

   (d_human, X , X ) → R_  → Primitive Weights
   Loss: Cross-entropy with successful synthesis primitives

   Phase 3: Joint fine-tuning

   X , X  → G_  → d → R_  → Weights → Enumeration → *
   Loss: Synthesis success (RL or differentiable relaxation)
```

3.2 Evaluation Strategy

1. **Hold out 10 rules** from training
2. **Train G_ and R_** on 35 rules
3. **Evaluate:**
   - Can G_ produce sensible descriptions for held-out rules?
   - Does R_ + G_ outperform R_ alone on held-out rules?
   - Are generated descriptions interpretable to humans?

3.3 Ablation Studies

Condition	What it tests
R_ only (no language)	Baseline—is language helping?
R_ + human descriptions	Upper bound—best possible guidance
R_ + G_ (random init)	Does structure of G_ matter?
R_ + G_ (pre-trained)	Does linguistic pre-training help?
R_ + G_ (joint training)	Does end-to-end optimization help?

Part IV: Connection to Self-Explanation Theory

4.1 Mapping to Chi et al. (1994)

Chi identified three mechanisms by which self-explanation aids learning:

Chi's Mechanism	Computational Analog
Inference generation	G_ infers discriminative features from examples
Mental model repair	Descriptions reveal mismatches → update R_
Coherence checking	Description must be consistent with observations

4.2 Why Self-Generated > Received

The self-explanation literature shows that **generating** explanations is more effective than **receiving** them. Possible computational reasons:

1. **Alignment**: Self-generated descriptions are optimized for the learner's own inference process
2. **Active engagement**: Generation requires deeper processing than passive reception
3. **Error detection**: Attempting to verbalize reveals gaps in understanding

4.3 The Language of Thought Connection

Your proposal connects to Fodor's "Language of Thought" hypothesis: - Internal representations have linguistic structure - Thought is compositional like language - Description generation = translating internal states to natural language

The self-description architecture makes this translation explicit and trainable.

Summary

Question 1: Why can't language compensate for missing primitives?

Answer: Language guidance affects primitive *selection*, not *composition*. Even with perfect primitive weights, enumeration must search through O(b^k) compositions where k is program depth. This is an exponential blowup that no amount of probability boosting can overcome.

Question 2: Can description generation be trained independently?

Answer: Yes, with careful design: - Pre-train G_ on language to ensure linguistic structure - Use contrastive learning to capture discriminative features - Fine-tune jointly for synthesis-specific alignment

This approach models **self-explanation**—the cognitive process of verbalizing observations to guide inference—and is more realistic than LAPS (human descriptions) or LILO (LLM descriptions).

Generated as part of /research-ultrathink analysis December 2024